🎯 Objective

To understand the limitations of a basic multithreaded client-server chat system and incrementally improve it into a simple multi-client chatroom.

📘 Background

(Server-client (multiple) communication using multithreading was covered in Lab 3)

You are given a Java-based multithreaded server and client program where:

- The server accepts multiple clients using threads.
- Each client communicates with the server individually.
- The server operator manually types replies.

However, this system has several design issues:

- It is not a true chatroom.
- Multiple threads share the same console input.
- Messages are not shared between clients.

⚠️ Problem Statement

The current implementation behaves like multiple one-to-one chats instead of a group chat. Your task is to modify the system step-by-step to make it more practical and closer to a real chatroom.

🧩 Tasks

🔹 Task 1: Observe the Current Behaviour

- Run the server.
- Connect multiple clients.
- Send messages from different clients.

Answer the following questions:

- Can clients communicate with each other?
- What happens when multiple clients send messages at the same time?
- Is the server operator able to reply efficiently?

🔹 Task 2: Remove Server-Side Manual Input

Modify the server so that:

- It does not use `BufferedReader(System.in)` anymore.
- The server does not manually type responses.

Hint: Remove or comment out: br.readLine();

🔹 Task 3: Store Connected Clients

Add a shared list in the server to keep track of all connected clients.

Hint: static ArrayList<ClientHandler> clients = new ArrayList<>();

- Add each new client to this list when they connect.

🔹 Task 4: Broadcast Messages

Modify the `ClientHandler` so that:

- When a client sends a message, it is forwarded to all other clients.

Expected behaviour:

Client 1 says: Hello
Client 2 receives: Client 1: Hello
Client 3 receives: Client 1: Hello

🔹 Task 5: Handle Client Exit

Update the code so that:

- When a client types `"exit"`, the connection closes properly.
- The client is removed from the shared list.

🔹 Task 6: Improve Client Program

Modify the client so that:

- It can send and receive messages simultaneously.

Hint:

- Use one thread for sending.
- Use another thread for receiving.

🔹 Task 7: (Optional Bonus) Add Client Names

Instead of using client numbers:

- Ask the user to enter a name when the client starts.
- Display messages like:

Alice: Hello
Bob: Hi!

✅ Expected Outcome

After completing the tasks:

- Multiple clients can connect.
- Messages from one client are visible to all others.
- No manual server input is required.
- The system behaves like a simple chatroom.

📌 Submission Requirements

- Updated `Server.java`
- Updated `Client.java`
- A detailed report including:
  - Observations from Task 1
  - Detailed explanation of changes made
- A short screen-recorded video demonstrating correct output

📌 Submission Format

- `Server_ID.java` (e.g., `Server_222311002.java`)
- `Client_ID.java` (e.g., `Client_222311002.java`)
- `Report_ID.pdf` (e.g., `Report_222311002.pdf`)
- `Output_Video_ID.mp4` (e.g., `Output_Video_222311002.mp4`)

Compress all files into a single ZIP file:
CSE_434_Assignment_ID.zip (e.g., CSE_434_Assignment_222311002.zip)

⚠️ Academic Integrity

You may use AI tools to assist in completing the tasks; however, the report must be written in your own words and demonstrate a clear understanding of the work. Any form of plagiarism will result in the submission being considered invalid.